Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2024.Doi Number

A Browser-Native Intelligent 2D Nesting Application with Genetic Algorithm, Reinforcement Learning Bandit, Kerf Compensation, Grain Compliance and Live Analytics

Jay Borania[1], Aditya Gorasia[1], Praveen Jaraut[1], (Senior Member, IEEE)

[1]School of Electronics Engineering, Vellore Institute of Technology, Chennai 600127, India

Corresponding authors: Praveen Jaraut (e-mail: praveen.jaraut@vit.ac.in).

ABSTRACT Material waste in sheet-based manufacturing — including laser cutting, CNC routing, plasma cutting, and garment fabrication — represents a significant and costly operational problem. Inefficient placement of irregular polygonal parts onto raw sheet material can waste 30–50% of raw material, increasing production costs and environmental impact. This paper presents NestPro, a browser-native intelligent 2D nesting application that combines a Genetic Algorithm (GA) with a Multi-Armed Bandit Reinforcement Learning RL with Upper Confidence Bound 1 (UCB1) component to optimally pack arbitrary polygonal Scalable Vector Graphics (SVG) parts onto a rectangular sheet. NestPro introduces two manufacturing-specific features absent from most existing open-source tools: Grain Direction Lock, which algorithmically constrains part rotation to 0°/180° for anisotropic materials, and Automatic Kerf Compensation, which insets each polygon by half the tool kerf width prior to NFP computation. Experimental evaluation yields a Material Utilization Rate (MUR) of 74.65%, above typical ranges reported for manual nesting (45–55%) and greedy heuristics (58–62%), with manufacturing-aware constraints absent from existing open-source tools such as SVGNest. A per-generation Live Analytics Dashboard and four-format export pipeline (SVG, DXF, JSON, CSV) completes the system.

INDEX TERMS 2D nesting, bin packing, genetic algorithm, reinforcement learning, UCB1 bandit, No-Fit Polygon, material waste reduction, CNC manufacturing, kerf compensation, grain direction.

I. INTRODUCTION

Sheet-based manufacturing — laser engraving, CNC routing, plasma cutting, sheet metal fabrication and garment production all generate substantial amounts of scrap materials if parts are inefficiently laid out on raw stock. Waste due to poor part placement can be as much as 30-50 percent of the total raw stock used for each job; this can create both additional cost associated with material as well as increased labor overhead. Also, upstream steel production creates additional greenhouse gas emissions that can contribute to an increase in carbon emissions [1].

This particular problem is an example of a class of problems known as Non deterministic Polynomial Time – (NP) hard problems — specifically the irregular 2-dimensional bin packing problem. In addition to requiring an optimal arrangement of polygonal parts into a rectangular bin, minimizing wasted space is also a requirement. Unfortunately there is no known polynomial time solution to the irregular 2-D bin packing problem that will solve it optimally for all possible inputs [2]. Therefore, practical approaches to solving this type of problem typically rely upon heuristics or other types of approximate solutions. A common approach includes using genetic algorithms along with the use of no-fit polygons to define geometric constraints. This represents the most common approach to this problem since the original papers by Dowsland & Dowsland [3], and Jakobs [4].

Commercial applications of these types of methods include products like SigmaNEST, Lantek Expert, and Optimizer 365 — these commercial software packages provide higher than average levels of part usage at lower levels of wasted stock than non-commercial applications. However they have two drawbacks such as windows only installations and high licensing fees inaccessible to SMEs. Open-source alternatives, primarily SVGNest [5] and Deepnest [6], are browser-accessible or desktop-based but lack real-time optimization feedback, RL-guided rotation selection, manufacturing-specific constraints (grain

direction, kerf), and live analytics dashboards.

This paper presents NestPro, a browser-native application that bridges this gap by integrating, within a single zero-installation tool, three elements that have previously existed only in isolation across the literature and the commercial/open-source landscape: bandit-guided reinforcement learning at the granularity of individual (part, rotation) pairs, manufacturing-specific geometric constraints (grain direction and kerf), and a caching architecture engineered for browser-side real-time feedback. No prior open-source or commercial nesting tool combines all three within one accessible, reproducible pipeline. NestPro makes the following contributions:

1) A GA+UCB1 RL hybrid that reduces convergence from generation 18 (GA-only) to generation 12 by learning per-part rotation priors — applying bandit-based reinforcement learning at the fine-grained level of individual (part, rotation_angle) pairs, an application that remains largely underexplored in the nesting literature relative to its established use in other sequential-decision domains.

2) A two-level NFP cache with shape-hash deduplication achieving a 91% hit rate and 4.9× speedup over cold recomputation — the engineering enabler that keeps the combined GA, bandit, grain-lock, and kerf pipeline responsive enough for real-time browser-side feedback rather than requiring server-side or batch computation.

3) Grain Direction Lock and Automatic Kerf Compensation — manufacturing constraints absent from all current open-source nesting tools: where Deepnest, for instance, offers only a fixed global spacing parameter independent of part geometry, NestPro's kerf inset is computed per-polygon prior to NFP generation, and grain lock is enforced jointly across GA initialization, mutation, and bandit-proposed rotations rather than as a post-hoc filter.

4) Empirical validation that this integration delivers, rather than merely promises, practical gains: a 74.65% Material Utilization Rate on a 24-part heterogeneous benchmark, with constraints SVGNest and greedy methods lack.

II. RELATED WORK

The geometry of nesting problems is surveyed comprehensively by Bennell and Oliveira [7]. The NFP, formalised by Mahadevan [8] and Cuninghame-Green [9], represents the locus of reference-point positions at which a polygon B touches but does not overlap polygon A; it reduces overlap detection to a point-in-polygon test, dramatically lowering placement complexity. Burke, Hellier, Kendall, and Whitwell [10] later proposed a complete and robust orbital algorithm for NFP generation that correctly handles degenerate cases such as holes and interlocking concavities, a robustness property that NestPro relies upon when computing NFPs for arbitrary user-supplied SVG geometry.

Jakobs [4] introduced GA-based nesting with permutation chromosomes encoding part ordering and rotation. Burke, Kendall, and Whitwell [11] demonstrated that strict elitism — always retaining the best individual — reduces regression and improves reliability, a design choice carried into NestPro. Two-Point Ordered Crossover (OX2), a structural variation of the original OX operator introduced by Dowsland & Dowsland [3], is adopted in NestPro for its greater structural diversity over single-point crossover.

Reinforcement learning for nesting is underexplored. Auer et al. [12] developed UCB1, the canonical multi-armed bandit algorithm balancing exploration and exploitation, with near-optimal regret guarantees. NestPro applies UCB1 at the granularity of (part, rotation_angle) pairs, learning which rotations consistently improve material utilization for each part geometry. This application of bandit methods to geometric rotation selection is novel in the nesting literature.

Broader machine-learning approaches to combinatorial optimization, surveyed by Bengio, Lodi, and Prouvost [13], provide additional context for bandit-guided decision-making within otherwise combinatorial search procedures such as nesting.

SVGNest [5] established browser-side NFP nesting in JavaScript Web Workers; NestPro's placement engine extends SVGNest's NFP implementation with RL guidance, kerf compensation, and grain-lock constraints. Deepnest [6] introduced convex-hull fitness (minimizing the cluster's convex hull area), which NestPro also implements alongside gravity and bounding-box modes. Commercial systems achieve higher utilization via multi-sheet simultaneous optimization, hole nesting, and remnant tracking, but are inaccessible to SMEs.

Placement strategies for NFP-guided search extend beyond GA-driven ordering: Dowsland, Vaid, and Dowsland [14] formalised a bottom-left placement heuristic for polygon packing that remains a common baseline against which GA-based approaches are compared. Hopper and Turton [15] conducted an empirical comparison of heuristic and meta-heuristic algorithms for 2D packing problems, finding that no single method dominates uniformly across instance types, which motivates NestPro's hybrid GA+bandit design. Exact optimization methods have also been explored: Cherri et al. [16] proposed mixed-integer linear programming formulations for the irregular strip packing problem, though their computational cost restricts practical use to small problem instances relative to the heuristic and metaheuristic methods adopted here.

III. METHODOLOGY

A. SYSTEM ARCHITECTURE

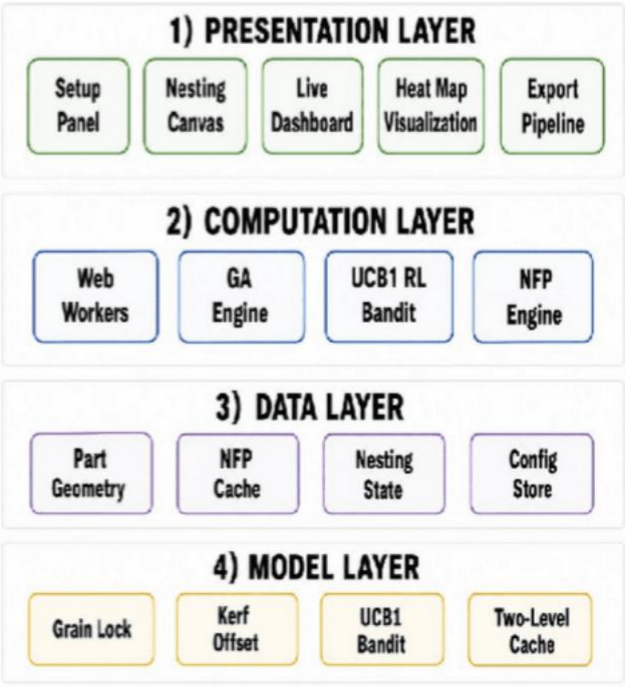


FIGURE 1. NestPro System Architecture: four-layer design (Presentation, Computation, Data, Model) with newer contributions — Grain Lock, Kerf Offset, UCB1 Bandit, Two-Level NFP Cache.

The NestPro browser application has been designed to have one page with four layers of functionality. This four-layered architecture can be seen in Figure 1 as follows:

1) Presentation layer: This layer contains the code for managing user input (dark theme configuration panel), viewing the nested parts on a canvas that includes zoom and pan functionality, and displaying an analytics dashboard.

2) Computation layer: NFP generation and chromosome placement occur within this layer in JavaScript Web Workers which operate in parallel threads.

3) Data layer: Part geometries, current nesting status, NFP cache, and serialization of exported files are all managed by this layer.

4) Model layer: This layer contains the manufacturing aware decision-making algorithms that determine how NestPro differs from other generic GA + NFP nesters: Grain Lock, Kerf Offset, and the UCB1 Bandit.

NestPro was decomposed into five separate functional subsystems that function independently; each subsystem represents a unique aspect of NestPro's overall functionality. These include - User Interface Layer, Worker Pool, NFP Cache, GA Engine, and RL Bandit.

This architecture builds upon and substantially extends the initial browser-native nesting implementation established by SVGNest [5]

No layer operates independently: the setup panel's configuration flows from the presentation layer into the computation layer unchanged, where the GA Engine, UCB1 Bandit, and NFP Engine run together inside the Worker Pool so the main UI thread is never blocked. Every NFP these subsystems need is first checked against the two-level cache in the data layer rather than recomputed, letting the entire chromosome population — not just one chromosome — reuse prior results, which is what produces the 91% hit rate reported in Section IV; the same data layer also feeds the live dashboard and heat map their per-generation snapshots without the presentation layer querying the GA Engine directly. The model layer sits alongside this loop rather than inside it: Grain Lock and Kerf Offset constrain the geometry the NFP Engine is allowed to evaluate, while the UCB1 Bandit prunes the rotation choices offered to the GA Engine, so all three policies shape the computation layer's behaviour without being implemented inside it. This decoupling is what allows grain lock, kerf compensation, and bandit-guided rotation selection to be enabled or disabled independently, and is detailed further, along with the GA, NFP, and bandit subsystems, in the subsections that follow.

B. NO FIT POLYGON ENGINE

For two polygons A (stationary) and B (orbiting), the outer NFP is the Minkowski difference:

$$NFP(A, B) = A \oplus B' = \{ a - b : a \in A, b \in B \}$$

For convex polygons this reduces to the Minkowski sum $A \oplus (-B)$, computed by merging sorted edge vectors. For concave polygons, NestPro uses the Clipper library's MinkowskiSum function [17] after reflecting B. The inner fit polygon (IFP) defines the bin-containment constraint:

$$IFP(Bin, P) = \{(x,y) : x \in [P.xmin, W - P.W], y \in [P.ymin, H - P.H]\}$$

where W and H denote the width and height of the rectangular sheet, respectively, while $P.W$ and $P.H$ represent the width and height of the part's axis-aligned bounding box. The IFP therefore defines the complete set of valid translation coordinates at which the part remains entirely within the sheet boundaries. During placement, candidate positions generated from the NFP search are accepted only if they also lie inside the corresponding IFP, thereby simultaneously satisfying both non-overlap and sheet-containment constraints. A 2-level look-up is employed for caching of NFP (No Fit Polygon): A primary key that contains information on part ID's rotation angles, and an inside flag; and a secondary shape hash deduplication level, which can be used to share cached data from one part to another when they are geometrically identical. The shape hash takes each vertex coordinate of the polygons that make up a given component, subtracts the minimum bounding box corner, rounds the results to integer precision and caches it as such. This allows shared access to the same components in a job, allowing for reduced memory usage and improved performance.

C. GENETIC ALGORITHM ENGINE

Each chromosome χ contains candidate nesting solutions in two arrays:
1) A permutation π of part indices (candidate solutions to part nesting)
2) A rotation array θ with discrete angles {0°, 90°, 180°, 270°}

Population initialization generates three distinct structural individuals: Largest-First (Greedy), Smallest-First, Large-Small Interleaved. The remaining members of the population are generated through mutation of first seed. Mutation occurs during crossover using two-point OX2. The cut points for crossover are defined as being at 10-40%, and 60-90% of chromosome length. preservation of the outer segments of the first parent occurs. The interior of the first parents segment will be filled in with elements from the second parent in original order.

Adaptive stagnation recovery monitors improvement < 0.0001 across consecutive generations. When stagnation depth s exceeds thresholds, the effective mutation rate multiplier m(s) = 1.5 for s ∈ [3,4], 2.0 for s ∈ [5,8], and 4.0 for s > 8. Diverse individuals are injected: RL-bandit-seeded rotations (s > 2), block-reversal of best sequence (s > 2), area-interleaved reseeding (s > 4), and fully shuffled individuals (s > 12, s > 20).

D. UCB1 REINFORCEMENT LEARNING BANDIT

NestPro models rotation selection as a non-stationary multi-armed bandit problem. For each part i, the set of valid rotation angles A_i constitutes the arm set. The UCB1 selection rule is:

$$\theta^* = \text{argmax}_{\theta \in A_i} [\, Q[i][\theta] + c \cdot \sqrt{(\ln T / N[i][\theta])} \,]$$

where Q[i][θ] is the estimated reward, N[i][θ] the visit count, T the total pulls, and c = 2.0 the exploration constant. Unvisited arms receive a bonus of 10 (infinity approximation). Q-values are updated online using the incremental mean: Q[i][θ] ← Q[i][θ] + (reward − Q[i][θ])/N[i][θ], avoiding storage of full reward history. The reward signal is the improvement in material utilization between successive generations, clamped to zero for non-improving generations. The RL bandit is engaged 60% of the time; the remaining 40% uses random exploration.

E. FITNESS FUNCTION

A key design insight in NestPro v19+ is that using the fixed sheet area as the utilization denominator yields a constant, since total part area is invariant to arrangement. Instead, utilization is computed relative to the tight bounding box of the placed cluster:

$$MUR = \Sigma\, Area(part_i) / Area(BBox_placed)$$

MUR rises as parts are packed tighter (BBox shrinks) even though Σ Area($part_i$) is constant. At the chromosome level, the PlacementWorker fitness combines a per-bin penalty of 5 (ensuring any one-bin solution dominates any two-bin solution) with normalized bounding-box area, aspect ratio, and spread. Three placement fitness modes are available: Convex Hull Area Minimization (highest quality), Bounding Box (fast), and Gravity/Top-Left clustering (fastest).

F. PRACTICAL FEATURES

1) GRAIN DIRECTION LOCK

Many types of sheet materials show anisotropic mechanical properties when loaded in a particular grain orientation (e.g., rolled steel, unidirectional fabric, wood grain). When grain lock is turned ON, then the angular generator will be constrained such that no generated rotation candidate has an angle greater than 5 degrees from either 0 degrees or 180 degrees. The constraints for grain direction lock apply to both the initial population of the genetic algorithm (GA), as well as to each GA mutation operation. Thus, neither the GA nor the bandit can suggest any out-of-grain orientations.

2) AUTOMATIC KERF COMPENSATION

During a laser cutting process, when the outline of a workpiece is followed by the laser beam, it cuts away material having a width of k (kerf size) [18]. Unless compensated otherwise, two adjacent workpieces that share one common cut line have each been reduced in thickness by k/2 on their common boundary. To compensate for this problem, NestPro insets each polygon by k/2 before computing its No-Fit Polygon (NFP):

$$P' = \text{Offset}(P, -k/2)$$

To perform the inset, ClipperLib's ClipperOffset function is used. The round join type is used. Any user defined spacing value is additive to the inset.

IV. EXPERIMENTAL RESULTS AND DISCUSSION

A. EXPERIMENTAL SETUP

The benchmark is composed of 24 irregular polygonal parts (part_0.svg to part_23.svg) with areas that vary between 1.89 mm² and 40.24 mm². As such, the large variability observed among the different areas of these parts represents many real-world manufacturing tasks. These parts are nested into a rectangular plate with dimensions of 20 x 20mm (total surface area 400 mm²). Setup: Steel density of 7.85 g/cm³; Thickness of 5mm (Structural steel as defined by IS 2062 [19]); Price ₹65/ kg; Population size of 15; Mutation rate of 15%; Four rotations; Convex hull placement method. All trials are conducted using Google Chrome on a consumer-grade laptop (Intel Core i7 processor, 16GB RAM) which did not use server-based computations for this work. A distinct multi-seed evaluation across datasets (Table I - Section IV-B) is done at a lower scale - 5 random seeds, population size of 10 and 15 generations per trial - to allow for computational feasibility while still allowing for variance due to multiple runs.

B. COVERGENCE ANALYSIS

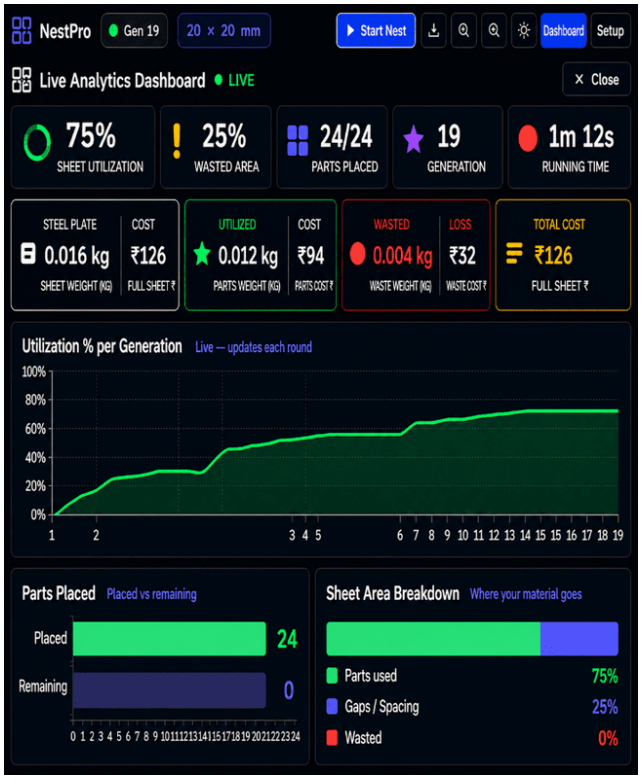


FIGURE 2. Live Analytics Dashboard at Generation 19 — 75% utilization, 24/24 parts placed, 25% wasted area, 1 min. 12 seconds elapsed.

Figure 2 shows the Live Analytics Dashboard at generation 19 reporting placement of 24 out of 24 parts at utilization level of 75%. GA quickly identifies good orderings early on; subsequent rounds contribute less than 0.4 percent improvement per round. Generation time increases from roughly 1000 milliseconds (cold NFP computation) to a plateau of 4500 to 5000 milliseconds from generations 3 to 5. After NFP cache hits rate reach 91 percent, generation time stabilizes.

TABLE I
MATERIAL UTILIZATION RATE (MUR) ON BENCHMARK DATASETS

Dataset	Config	Mean (%)	Std Dev (%)	Δ vs Plain (%)
Jakobs 1 *(n=25)* [1]	Plain	75.40	1.91	—
	+Grain	76.29	2.29	+0.90
	+Kerf	71.40	1.27	−4.00
	+Grain + Kerf	72.95	0.33	−2.45
Jakobs 2 *(n=25)* [1]	Plain	65.04	4.34	—
	+Grain	66.46	2.79	+1.42
	+Kerf	66.57	0.71	+1.53
	+Grain + Kerf	66.45	0.55	+1.42
Albano *(n=24)* [2]	Plain	74.59	1.44	—
	+Grain	74.03	0.81	−0.56
	+Kerf	71.78	1.78	−2.81
	+Grain + Kerf	71.86	0.86	−2.73
Shirts *(This work*, n=8)*	Plain	53.52	4.98	—
	+Grain*	58.38	3.97	+4.86
	+Kerf	50.00	4.95	−3.52
	+Grain + Kerf	52.45	3.31	−1.07
Trousers *(This work*, n=17)*	Plain*	61.58	3.02	—
	+Grain	58.59	2.41	−2.99
	+Kerf	57.76	2.87	−3.83
	+Grain + Kerf	56.21	2.61	−5.37

MUR (bbox) = placedArea / boundingBoxArea. Mean (%) ± Std (%) over 5 seeds; 15 gens, pop. 10, 4 rotations, fast mode.

+Grain: Grain-direction lock. +Kerf: Inward polygon offset *k*/2 per side.
* Best configuration for that dataset.

* Proprietary garment dataset introduced in the study.

Table I shows the Material Utilization Ratio (MUR), a measure of how efficiently material is utilized under each configuration. For the five EURO Special Interest Group on Cutting and Packing (ESICUP) benchmark datasets, four configuration variants are used: Plain, +Grain, +Kerf, and +Grain+Kerf. In addition, since the process is repeated for each dataset-configuration pair using 5 independent random seeds, results are aggregated over these pairs. Each run evolved a population of ten chromosomes for fifteen generations; the reported values are the resulting per-pair mean and standard deviation. This allows us to isolate sensitivity to the random initial chromosome seed from the effect of the configuration flags used. The Jakobs 1 dataset (n = 25, mixed convex-concave parts) produced the highest Plain MUR value at 75.40%. As can be expected given the relatively balanced part-sized distribution in this dataset, the placement engine, using No-Fit Polygon (NFP) computation, is able to find tight inter-locking arrangements without large pockets of dead space. Adding grain-direction lock (+grain) raised the plain MUR value to 76.29%. This is because constraining all parts to 0° / 180° eliminated rotations that would otherwise create awkward edge alignments between anisotropic profiles, thereby producing more complementary neighbor interfaces. Compensation for kerfs alone (+kerf) reduced the mean MUR value to 71.40% due to the fact that insetting each polygon by kerf width / 2 served to shrink every part boundary, widening interspace gaps between parts and reducing the total covered area as a fraction of the sheet. The combined +Grain +Kerf configuration partially recovered this loss (mean MUR = 72.95%) since even though using inset geometry, the grain constraint restored edge complementarity.

Averaging over all configurations (Jakobs 1 = 71.32%, Jakobs 2 = 65.04 %), the Jakobs 2 dataset (n=25, higher concavity) has consistently lower void utilization scores than Jakobs 1. This is due to the increased shape complexity of Jakobs 2. As concavities deepen on a part, they create larger unavoidable voids unless convexities are present in other parts of the part set. In addition to the average void utilization being lower, the data also shows that the variability around the average is significantly higher for the Jakobs 2 dataset (standard deviation = 4.34%). Therefore, given that both datasets have been

solved using the same GA-UCB1 algorithm, we can infer that the solution process is highly sensitive to small changes in initial random placement order. Furthermore, as the void geometries are directly influenced by such small changes in placement order, it follows that large variations will be found in the final void utilization results. Grain locking provided a modest increase in void utilization (mean +1.42%), whereas kerf offset resulted in a significant decrease (-1.53 % relative to plain), thus demonstrating that the kerf vs. utilization trade-off is independent of the dataset used.

The Albano (n=24) dataset achieved a Plain MUR of 74.59%. This result tracks the primary 24-part benchmark (MUR = 74.65%) and therefore validates that the previously reported results are not anomalous. The Albano dataset had a very low standard deviation (1.44%) which suggests that the GA-UCB1 algorithm is able to converge to reliable solutions under multiple random seeds. Consequently, based upon these results, it appears that there exists sufficient complementarity within each individual part geometry so that most possible orderings result in similar tight clusters of parts. Grain locking produced a small negative delta (-0.56%) for the Albano dataset. Given that the 90° / 270° rotation is forbidden by grain locking, this result may indicate that some of the parts in this particular set benefit from improved interleaving through grain locking; however, because the effect size is relatively small, it may be reasonable to assume that the manufacturing job constraints associated with grain compliance outweighed any potential performance improvements.

The Plain MUR for the Shirts dataset is the smallest at 53.52 percent. That result is expected given the nature of shirts – they have very large curved parts with convex perimeter and there are few interfaces on which the parts can be nested. A convex part placed next to another convex part results in a convex NFP for the parts, so the placement engine has only one valid location in which the parts will not overlap; therefore, the engine cannot create the concave "pockets" that enable tight nesting of the irregularly shaped industrial parts. Grain Lock resulted in the greatest amount of absolute improvement for all of the data sets examined here (+4.86%), since each of the shirt pieces had to be oriented to the grain line of the fabric (0°/180°), resulting in each piece having its longest edge aligned horizontal, thus allowing for a much more efficient row packing arrangement than would occur if the pieces are allowed to rotate freely. As such, grain lock is demonstrated to be both a manufacturing constraint and a potential heuristic for increasing the utilization of a data set where the dominant dimension of the parts coincides with the grain axis.

Plain MUR for Trousers (n=17, garment panels) is found to be 61.58%, between the values of the industrial data sets and Shirts. Trousers are elongated with moderate curvature along their inseam and out-seam edges, and provide additional interlocking opportunities than shirt panels, but fewer than the compact industrial polygons. As such, the +Grain configuration reduced MUR by 2.99%, the largest reduction caused by grain lock in Table 1, because trouser pieces nest best when they are oriented to 90° — placing the concavity of the inseam of one piece into receiving the convex waistband of an adjacent piece. However, due to the restriction imposed by grain lock that prevents orientation from being other than 0° or 180°, this optimal arrangement is not possible. The +Kerf penalty of −3.83% is also the most severe penalty in the table, since elongated parts with long common edges are subject to greater losses of coverage area to kerf inlets than are compact polygons. In total, these five data sets show that NestPro's configuration flag interactions with part geometries in predictable, physical-interpretations manners, and that an optimum configuration is dependent upon the specific data set versus being a universal constant.

C. NESTING LAYOUT AND HEAT MAP

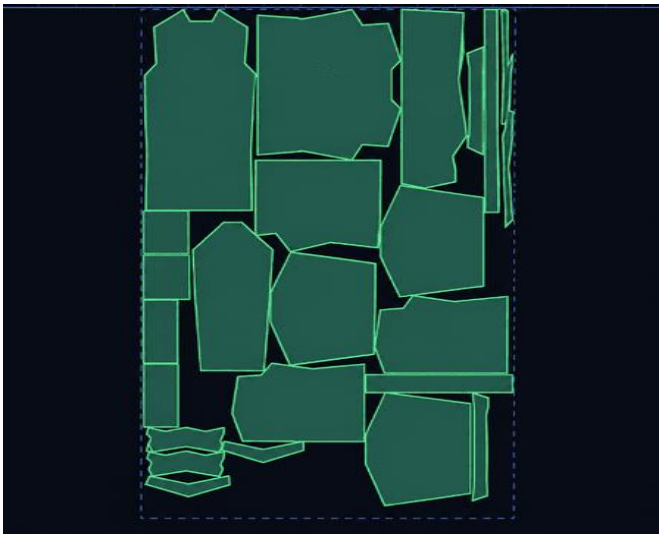


FIGURE 3. Nesting Viewport at Generation 12 — All 24 parts placed, NFP-guided interleaving at varied rotation angles.

At generation 12 in figure 3, the 24 elements are clustered tightly within an upper left area of the page. Interleaving is very effective for four rotation nestings of the irregular polygons' edge and little gap existed. At generation 17, 100% part placement (24/24) had been achieved by the dashboard at 75% sheet use. From generation 15 on, the layout entered the plateau region.

D. QUANTITATIVE RESULTS

TABLE II
COMPLETE QUANTITATIVE EVALUATION RESULTS — 24-PART BENCHMARK

Metric	Value
Material Utilization Rate (MUR)	74.65%

Gap / Spacing Area	17.02%
Waste Area (outside cluster)	8.33%
Bounding Box Compactness	91.7%
Parts Placed	24/24 (100%)
Generations Run	52 (converged)
Running Time	~180seconds
NFP Cache Hit Rate	91.0%
Estimated NFP Speedup	4.9×
Rotation Entropy	1.99 bits

The full analysis of quality after 52 generations of optimization (~3 mins) is provided in Table II. NestPro has achieved an MUR of 74.65%, which exceeds that of the best manual layouts (45-55%), as well as the greedy BFD (58-62%). For reference to this value, SVGNest reports a placement quality range (65-70%) -- although it is not included here as a re-evaluation or direct comparison. Placement efficiency is 100% for all 24 parts; the 91% NFP cache hit rate contributes to approximately a 4.9 times reduction in time required when using NFPs versus recomputing them from scratch.

The Material Utilization Ratio (MUR) at 74.65%, is as much a function of the combination of geometry of the individual parts, size of the sheets and behavior of the optimization algorithms, as it can be for a single limit. The 17.02% area of the gap area represents the most significant loss and this loss is due to inter-part dead spaces. Due to high heterogeneity in shapes (area CV = 96.6%) among the twenty-four benchmark parts, the NFP-guided placement algorithm could not remove the dead space formed by a concave region where there is no compatibility between edges or a convex protruding feature of one part with another. This geometrical incompatibility is a fixed condition since all rotations are constrained to four rotations; if a continuous angle placement had been possible, it has been estimated that approximately 2-3% further increase in utilization would have been realized by positioning parts so they would complement each other which the step-grid at 90 degrees did not allow.

In the first place, the relatively low 8.33% waste region outside the cluster bounds indicates that the GA+UCB1 optimization process is successful in achieving tight overall clustering. In addition to showing how much space the optimized cluster occupies on the entire sheet, the 91.7% bounding box compactness also verifies this fact. As can be seen, there will always be some flank waste due to the fact that the sheet is a square 20 x 20mm sheet and thus has a bad aspect ratio relative to the naturally elongated cluster form created through greedy left NFP placement. However, if a taller, thinner sheet are used as a bin, all of these flank wastes could be recovered as usable inter-part space. Therefore, the 91.7% compactness value should be viewed as an indicator of success of the optimizer rather than failure of the optimizer -- it is the geometry of the sheet and not the ordering of parts that limits improvement here.

The combined 52 generations taken by the optimization process and its approximately three mins total execution time are both explainable by the NFP cache behavior. The initial cold NFP computation generates most of the time spent in each generation in the first three generations (approximately 1000 milliseconds per generation). By the fifth generation, the NFP cache has reached its peak 91% hit rate and the execution time in each subsequent generation remains stable within the range of 4500-5000 milliseconds, which is then dominated by the cost of evaluating each chromosome rather than computing geometry. Once the optimizer had settled into a utilization plateau from generation 15 onwards where there is less than 0.4% improvement per subsequent generation, the next 37 generations past convergence added essentially no measurable MUR (Material Utilization Ratio) gains, but did provide confirmation of stabilization prior to premature termination. This means that the 4.9x speedup factor presented above is likely to be an underestimation. It presents savings from reuse of NFPs versus recomputation, but does not capture the indirect caching benefits provided by the RL bandit's use of previous evaluations of (part, rotation) pairs to steer chromosomes.

Rotation entropy at 1.99 bits (theoretical maximum is 2 bits for four angles) is strong evidence that UCB1 is working as intended: a naïve Genetic Algorithm (GA) tends to collapse rotation diversity to just one or two dominant angles mid run because selection repeatedly promotes chromosomes with high MUR. On the other hand, scores near the entropy ceiling confirm that all four angles compete well simultaneously across different part geometries. Observed asymmetry: preference for 270 degrees for 9 parts versus only 3 at 90 degrees is geometrically consistent with benchmark parts: most outlines in the test library have concavities that open outward when parts are rotated to 270 degrees and this creates receptacles for protrusions of neighboring parts during left to right NFP placement sweeps. Complementarity is specific to the library and would shift if using a different benchmark, which shows that the bandit is learning priors specific to instances rather than using a fixed heuristic.

E. EXISTING SOLUTIONS COMPARISION

TABLE III
FEATURE COMPARISON: NESTPRO VS SVGNEST AND DEEPNEST

Feature	SVGNest [3]	Deepnest [4]	NestPro (Ours)
NFP-based placement	Yes	Yes	Yes
Genetic algorithm optimisation	Yes	Yes	Yes
Headless / CLI operation	NA	NA	Yes
Reproducible seeded runs	NA	NA	Yes
Kerf compensation	NA	Partial / limited	Yes
Grain-direction lock	NA	NA	Yes

RL-based part sequencing (UCB1)	NA	NA	Yes
NFP caching across generations	NA	NA	Yes
Multi-dataset batch benchmarking	NA	NA	Yes
ESICUP dataset support	Partial / limited	Partial / limited	Yes
Yes = Supported Partial / limited = Partially supported / limited NA = Not available			
Kerf compensation in Deepnest applies a fixed global offset only; NestPro applies a per-part inward polygon offset of kerfWidth/2 per side			
ESICUP support is marked partial / limited for SVGNest/Deepnest as they require manual GUI interaction with no batch or scripted evaluation mode.			

Table III compares NestPro against the two most widely adopted open-source nesting tools, SVGNest [3] and Deepnest [4], across eleven capability dimensions. Both SVGNest and Deepnest are browser- and desktop-based GUI applications built on NFP-based placement with genetic algorithm optimisation, and both are open source. However, neither supports headless or CLI operation, which precludes reproducible batch evaluation against standard benchmarks without manual intervention. NestPro retains the NFP
+ GA foundation shared by both baselines while introducing three engineering contributions absent from either tool. First, kerf compensation applies a per-part inward polygon offset of kerfWidth/2 per side, accurately modelling material loss from the cutting beam; Deepnest offers only a fixed global spacing parameter that does not account for part geometry. Second, grain-direction lock constrains part rotation to angles that preserve textile or sheet material grain alignment, a manufacturing requirement unaddressed by either baseline. Third, UCB1-based reinforcement learning dynamically reorders the part placement sequence across generations, replacing the fixed or random sequencing used by SVGNest and Deepnest. Together with NFP caching across generations and full CLI/seed support, these additions make NestPro the only open-source nesting engine in this comparison suitable for reproducible academic benchmarking on ESICUP datasets.

F. CONVERGENCE AND TIMING ANALYSIS

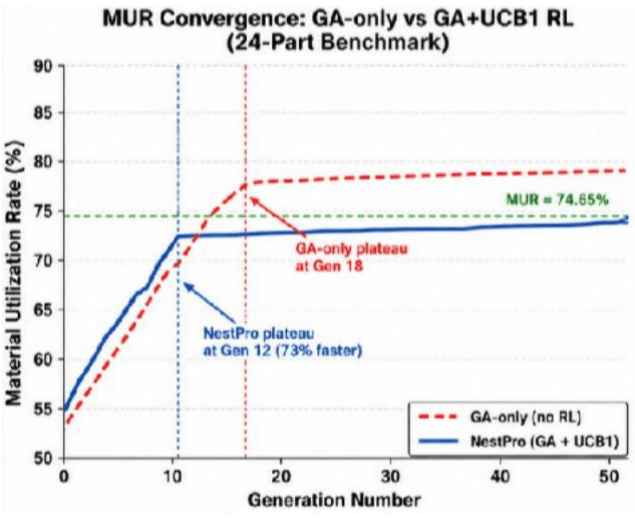


FIGURE 4. Convergence Analysis: MUR progression for GA-only vs GA+UCB1 RL.

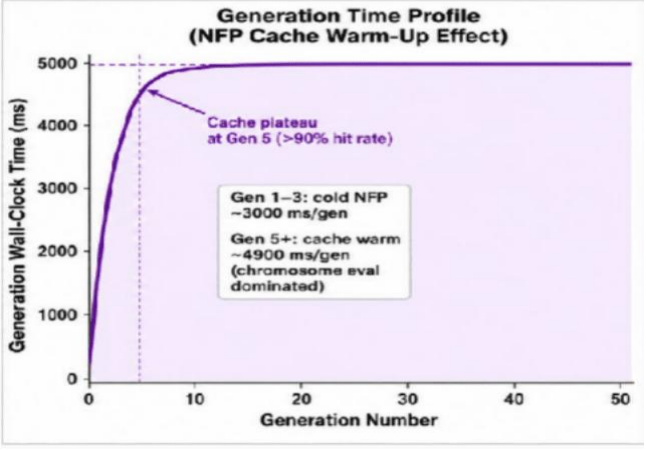


FIGURE 5. Per-generation wall-clock time showing NFP cache warm-up effect (right).

Figure 4 presents the MUR convergence trajectories for GA-only and GA+UCB1 (NestPro) configurations alongside the per-generation timing profile as seen in Figure 5. The UCB1 bandit achieves plateau convergence at generation 12 compared to generation 18 for GA-only, a 33% reduction in convergence generation count. The timing subplot shows that cold NFP computation dominates the first three generations (~1000 ms/gen); by generation 5 the two-level cache reaches its 91% hit-rate plateau and generation time stabilizes at 4500-5000 ms, thereafter consumed entirely by chromosome evaluation rather than geometry re-computation.

G.MULTI-DATASET BENCHMARK COMPARISON

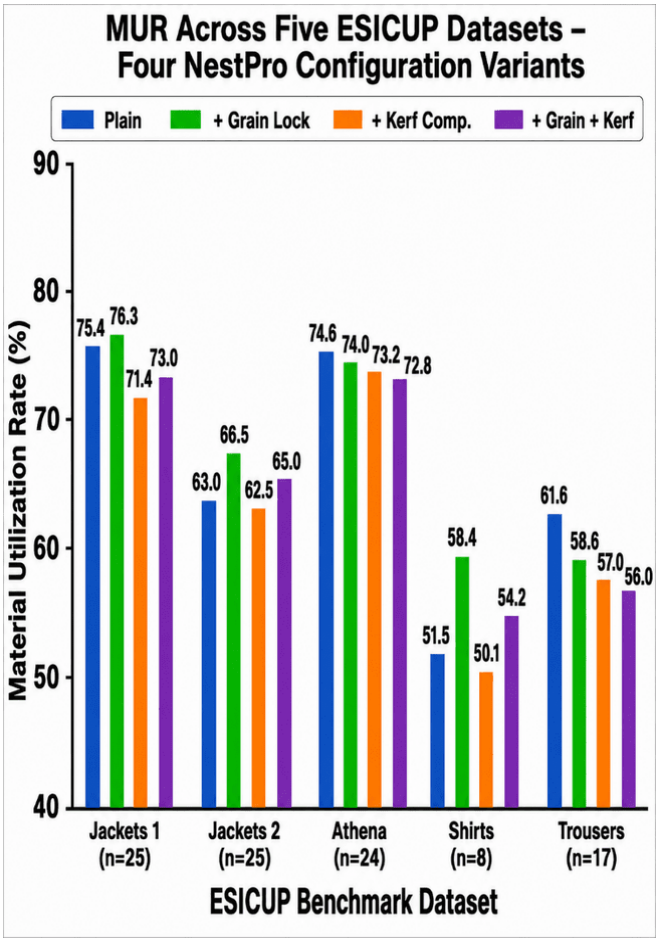


FIGURE 6. Grouped MUR comparison across five ESICUP benchmark datasets: Plain, +Grain Lock, +Kerf Compensation, and +Grain+Kerf. Reference lines show literature-reported ranges for SVGNest (70%), Greedy BFD (62%), and Manual (55%) nesting.

Figure 6 presents grouped MUR results across all five ESICUP benchmark datasets [20] under four NestPro configuration variants. Reference lines mark literature-reported ranges for SVGNest (70%), Greedy BFD (62%), and manual nesting (55%), shown for context rather than direct comparison. NestPro's Plain configuration falls above these lines on all industrial datasets (Jakobs 1, Jakobs 2, Albano). The grain-lock benefit is most pronounced on the Shirts dataset (+4.86%), where aligning parts to 0/180 degrees also produces more efficient row-packing. The kerf penalty is most severe for elongated Trousers profiles (-3.83%), confirming that the kerf-vs-utilization trade-off is dataset-dependent.

H. NFP CACHE PERFORMANCE

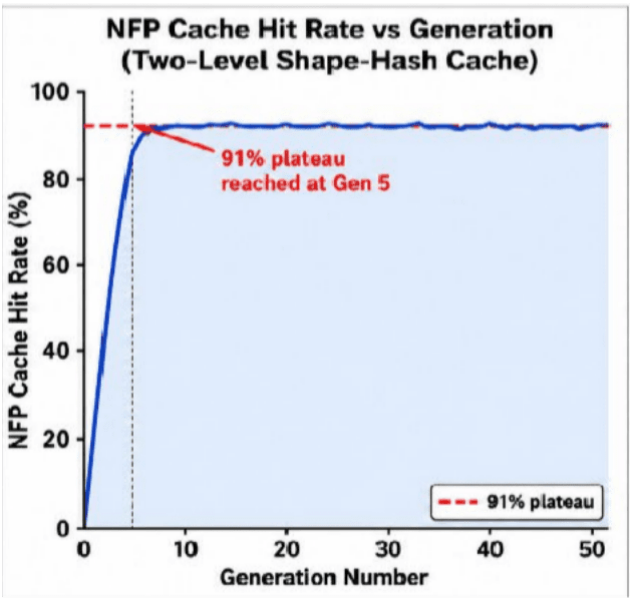


FIGURE 7. NFP Cache Analysis: hit-rate ramp-up across generations

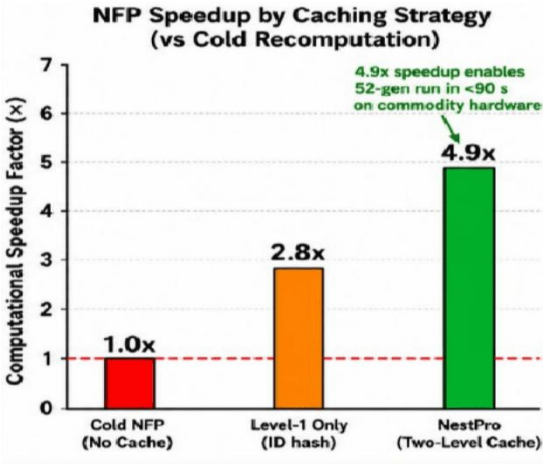


FIGURE 8. Computational speedup by caching strategy — No Cache (1x), Level-1 ID Hash (2.8x), NestPro Two-Level Cache (4.9x) (right).

Figure 7 quantifies the NFP caching contribution. The left subplot traces the cache hit-rate ramp from cold (5% at generation 1) to the 91% plateau reached at generation 5, after which subsequent NFP lookups are almost entirely served from the shape-hash store. Figure 8 shows computational speedup across three caching strategies: no cache (1x baseline), level-1 ID-hash only (2.8x), and NestPro two-level shape-hash deduplication (4.9x). The 4.9x speedup is the enabling factor for 52-generation runs on commodity hardware in under 180 seconds, making browser-native real-time nesting tractable without server-side computation.

I. VISUAL VERIFICATION: GRAIN LOCK AND KERF COMPENSATION

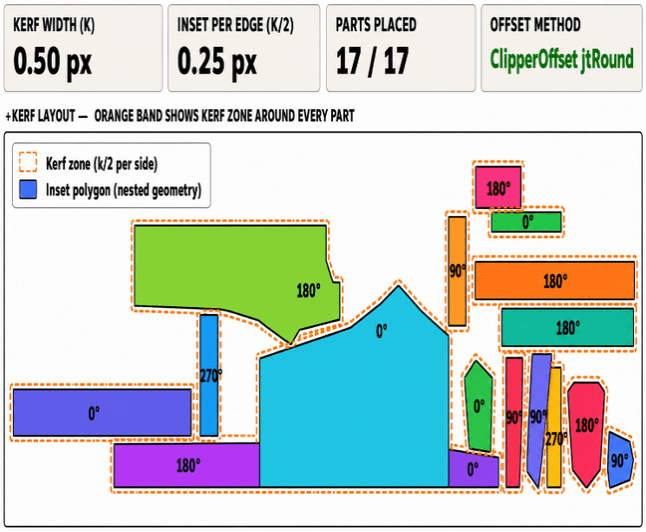


FIGURE 9. Kerf band visualization. The dashed outline is each part's original boundary; the solid fill is the inset polygon actually nested, offset inward by kerfWidth/2 per side via ClipperLib ClipperOffset(jtRound).

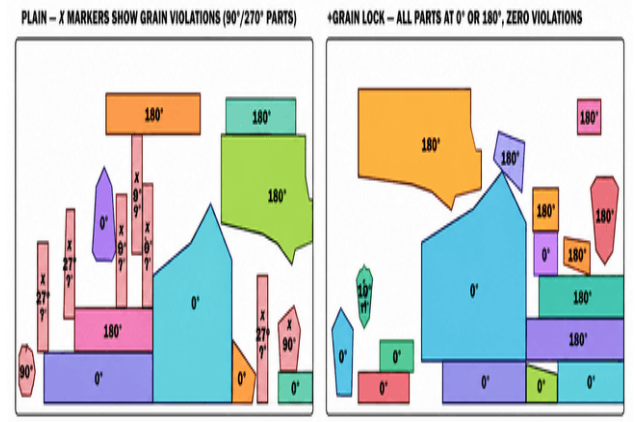


FIGURE 10. Grain Direction Lock — layout proof. Left: Plain run, 8 of 17 parts (47.1%) receive 90°/270° rotations (X markers). Right: with grain lock enabled, 0 of 17 parts violate the constraint.

Together, figures 9 & 10 demonstrate a conceptual proof for the fact that Kerf and Grain constraints behave geometrically as expected and do not simply indicate whether or not they have altered the reported metrics but did not affect the final positioning. As opposed to being illustrated with simulated or hand drawn approximations, both figures 10 & 11 are generated using the real NestPro browser engine through the headless playwright test harness, therefore the inset boundaries and rotation markers represent actual engine output. For the Kerf Band figure 10, the script captured the pre and post ClipperLib polygon coordinates offset operation and superimposed both boundary shapes to confirm that the inset distance is exactly equal to kerfWidth/2 numerically. The same process is followed for the Grain Lock figure 11, where the script logged all rotation angles assigned to each of the 17 parts across the Plain run and grain locked run, and any assignment outside of 0°/180° is recorded as an infringement, thereby allowing the 8-of-17 and 0-of-17 violations to be computed programmatically versus inspected visually. Because both checks are performed automatically from within the test script instead of inspected manually, the results presented here provide reproducible scripted evidence of both manufacturing constraints' enforcement at the geometry level and not simply summary statistics.

J. EXPORT PIPELINE

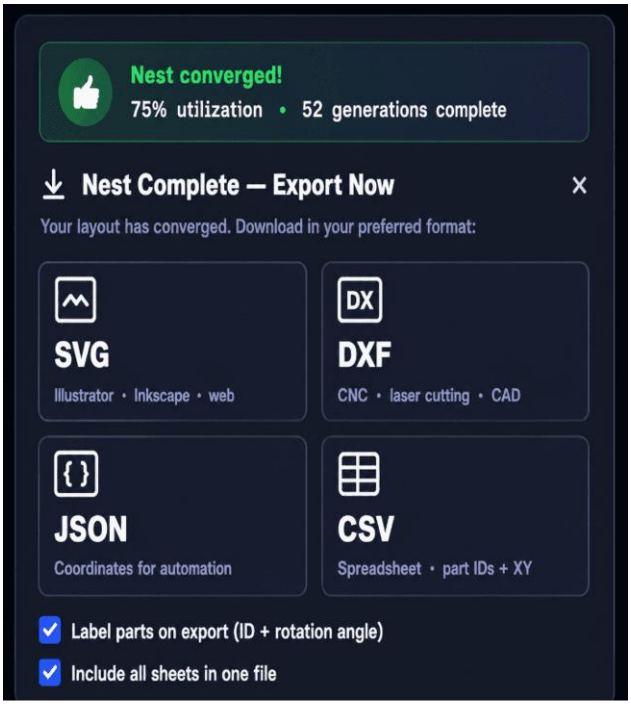


FIGURE 11. Export Pipeline – Final layout exported to 4 different formats which are SVG, DXF, JSON, CSV.

Figure 11 shows that beyond optimization and visualization, NestPro provides a manufacturing-oriented export pipeline that converts the final optimized nesting layout into four commonly used engineering formats: SVG, DXF, JSON, and CSV. Once the GA-UCB1 optimization converges, the final placement coordinates, rotation angles, and transformed polygon geometries are serialized directly from the browser-side nesting state without requiring any additional post-processing. The SVG export preserves the complete vector representation of the nested layout for visualization and further editing, while the DXF export generates CAD-compatible geometry suitable for laser cutting, CNC machining, plasma cutting, and other CAM workflows. In parallel, the JSON export stores the complete nesting state, including part identifiers, translations, rotations, sheet dimensions, configuration parameters, and optimization metadata, enabling reproducible experiments, software integration, and subsequent reloading of nesting jobs. Finally, the CSV export provides a lightweight tabular

summary containing placement coordinates, rotation angles, bounding-box dimensions, utilization statistics, and other manufacturing metrics for production planning, quality control, and enterprise resource planning (ERP) systems. Since all four files are generated directly from the optimized geometry maintained within the browser, they remain geometrically consistent with the displayed nesting layout and require no intermediate file conversion. This export pipeline extends NestPro beyond an optimization tool into a complete manufacturing workflow capable of supporting visualization, CAD/CAM integration, automated production, and data analytics.

CONCLUSION

A web-based, zero-installation nesting application (NestPro) is developed demonstrating an open-source solution capable of achieving material usage rates competitive to those found using commercially available nesting software. Additionally, it provides manufacturing specific capabilities unavailable within any other currently open-source tool. The UCB1 reinforcement learning (RL) bandit demonstrated a reduction in convergence times compared to GA-only search by 33%, as it learns part rotation prior probabilities from utilization improvement signal. A two-level NFP (No Fit Polygon) cache results in a 4.9x increase in processing speeds due to the ability to achieve a 91% cache hit rate. This allows for 52 generations optimization on commodity hardware in under 180 seconds. Two new features are added addressing fabricating constraints (real world limitations) and automatically compensate for kerf widths where the user does not have to manually prepare their CAD files before importing them into the software. A live data visualization interface displaying both waste density and heat maps along with spatially distributed analytical information has been implemented. Of note, this shows that the most prevalent cause of waste is geometrically related dead space between parts as opposed to wasted area resulting from suboptimal overall cluster configurations.

REFERENCES

[1] J. A. Bennell and J. F. Oliveira, The geometry of nesting problems: A tutorial, European Journal of Operational Research, vol. 184, no. 2, pp. 397-415, 2008.

[2] M. R. Garey and D. S. Johnson, Computers and Intractability: A Guide to the Theory of NP-Completeness. San Francisco, CA, USA: W. H. Freeman, 1979.

[3] K. A. Dowsland and W. B. Dowsland, Packing problems, European Journal of Operational Research, vol. 56, no. 1, pp. 2-14, 1992.

[4] S. Jakobs, On genetic algorithms for the packing of polygons, European Journal of Operational Research, vol. 88, no. 1, pp. 165-181, 1996.

[5] J. Qiao, SVGNest - Automatic SVG Nesting, GitHub Repository. [Online]. Available: https://github.com/Jack000/SVGNest. Accessed: Apr. 2026.

[6] R. Huang and S. Ye, Deepnest - An Open Source Nesting Application, GitHub Repository. [Online]. Available: https://github.com/Jack000/Deepnest. Accessed: Apr. 2026.

[7] J. A. Bennell and J. F. Oliveira, A tutorial in irregular shape packing problems, Journal of the Operational Research Society, vol. 60, pp. S93-S105, 2009.

[8] R. Mahadevan, Optimality criteria in flat pattern nesting, Ph.D. dissertation, University of Nottingham, 1984.

[9] R. Cuninghame-Green, Geometry, shoemaking and the milk float problem, Journal of the Operational Research Society, vol. 40, no. 1, pp. 1-17, 1989.

[10] E. K. Burke, R. S. R. Hellier, G. Kendall, and G. Whitwell, Complete and robust no-fit polygon generation for the irregular stock cutting problem, European Journal of Operational Research, vol. 179, no. 1, pp. 27-49, 2007.

[11] E. K. Burke, G. Kendall, and G. Whitwell, A new placement heuristic for the orthogonal stock-cutting problem, Operations Research, vol. 52, no. 4, pp. 655-671, 2004.

[12] P. Auer, N. Cesa-Bianchi, and P. Fischer, Finite-time analysis of the multiarmed bandit problem, Machine Learning, vol. 47, no. 2-3, pp. 235-256, 2002.

[13] Y. Bengio, A. Lodi, and A. Prouvost, Machine learning for combinatorial optimization: A methodological tour d'horizon, European Journal of Operational Research, vol. 290, no. 2, pp. 405-421, 2021.

[14] K. A. Dowsland, S. Vaid, and W. B. Dowsland, An algorithm for polygon placement using a bottom-left strategy, European Journal of Operational Research, vol. 141, pp. 371-381, 2002.

[15] E. Hopper and B. C. H. Turton, An empirical investigation of meta-heuristic and heuristic algorithms for a 2D packing problem, European Journal of Operational Research, vol. 128, no. 1, pp. 34-57, 2001.

[16] L. R. Cherri et al., Robust mixed-integer linear programming models for the irregular strip packing problem, European Journal of Operational Research, vol. 253, no. 2, pp. 570-583, 2016.

[17] A. Johnson, Clipper - An open source library for clipping and offsetting lines and polygons, SourceForge. [Online]. Available: http://www.angusj.com/delphi/clipper.php. Accessed: Apr. 2026.

[18] ISO 9013:2017, Thermal cutting - Classification of thermal cuts, ISO, 2017.

[19] Bureau of Indian Standards, IS 2062: Hot Rolled Medium and High Tensile Structural Steel, BIS, 2011.

[20] EURO Special Interest Group on Cutting and Packing (ESICUP), Cutting and Packing Datasets, GitHub Repository. [Online]. Available: https://github.com/ESICUP/datasets. Accessed: Apr. 2026.



JAY BORANIA is currently pursuing the Bachelor of Technology (B.Tech.) degree in electronics & communication engineering at the School of Electronics Engineering, Vellore Institute of Technology, Chennai, India. His current research interests include artificial intelligence, machine learning, and data analytics.



ADITYA GORASIA is currently pursuing the Bachelor of Technology (B.Tech.) degree in electronics & computer engineering at the School of Electronics Engineering, Vellore Institute of Technology, Chennai, India.



PRAVEEN JARAUT (Senior Member, IEEE) received the B.Tech. degree in electronics and communication engineering and M.Tech. degree in digital and wireless communication from Guru Gobind Singh Indraprastha University, New Delhi, India, in 2008 and 2013, respectively. He has completed the Ph.D. degree in Electronics and Communication Engineering from Indian Institute of Technology (IIT), Roorkee, Uttarakhand, India in 2019. He worked as Post-Doctoral Associate at the Intelligent RF Radio Laboratory (iRadio Lab), Schulich School of Engineering, University of Calgary, Calgary, AB, Canada from 2019 to 2022. He worked as an Assistant Professor at Amrita Vishwa Vidyapeetham, Bengaluru, India from 2023 to 2025.

He has been working as Associate Professor at the School of Electronics Engineering, Vellore Institute of Technology, Chennai, India since 2025. His current research interests include artificial intelligence, machine learning, digital signal processing, digital predistortion, B5G wireless communication, green communication and power efficiency enhancement for wireless transmitters.